```
import math
import random

def calculate_conflicts(state):
    """
    Calculates the number of conflicting pairs of queens (the cost/energy).
    A perfect solution has 0 conflicts.
    """
    conflicts = 0
    n = len(state)

    for i in range(n):
        for j in range(i + 1, n):
            # Same row conflict (impossible with our state representation)
            if state[i] == state[j]:
                conflicts += 1
            # Diagonal conflict: if row diff equals column diff
            if abs(state[i] - state[j]) == abs(i - j):
                conflicts += 1

    return conflicts

def get_neighbor(state):
    """
    Generates a neighbor state by randomly picking one queen
    and moving her to a different row in her column.
    """
    neighbor = list(state)
    col = random.randint(0, 7)
    row = random.randint(0, 7)

    # Ensure we actually change the row
    while row == state[col]:
        row = random.randint(0, 7)

    neighbor[col] = row
    return neighbor

def simulated_annealing_8queens(initial_temp, cooling_rate, max_iterations):
    """
    Solves the 8-Queens puzzle using Simulated Annealing.
    """
    # 1. State Representation: An array of 8 integers.
    # Index = Column, Value = Row position of the queen in that column.
    current_state = [random.randint(0, 7) for _ in range(8)]
    current_cost = calculate_conflicts(current_state)

    temp = initial_temp

    # 2. Optimization Loop
    for iteration in range(max_iterations):
        # If we hit 0 conflicts, we found a perfect solution!
        if current_cost == 0:
            return current_state, iteration

        # Generate a candidate solution by moving one queen
        neighbor = get_neighbor(current_state)
        neighbor_cost = calculate_conflicts(neighbor)

        # Energy difference (change in conflicts)
        # A negative delta means the neighbor has fewer conflicts (good)
        delta = neighbor_cost - current_cost

        if delta < 0:
            accept = True
        else:
            # Calculate acceptance probability for worse moves
            # (Allows escaping local minima early on when temp is high)
            acceptance_probability = math.exp(-delta / temp)
            accept = random.random() < acceptance_probability

        if accept:
            current_state = neighbor
            current_cost = neighbor_cost
```

```
        # Cool down the system
        temp *= cooling_rate

        # Guard against temperature dropping too low too fast
        if temp < 1e-6:
            temp = 1e-6

    return current_state, max_iterations

def print_board(state):
    """Helper function to print the final 8x8 chessboard visually."""
    for row in range(8):
        line = ""
        for col in range(8):
            if state[col] == row:
                line += " Q "
            else:
                line += " . "
        print(line)

# --- Configuration & Run ---
init_temp = 50.0
cool_factor = 0.995
max_steps = 50000

solution, steps = simulated_annealing_8queens(init_temp, cool_factor, max_steps)
final_conflicts = calculate_conflicts(solution)

print("--- 8-Queens Simulated Annealing ---")
print(f"Final State (Columns 0-7 Rows): {solution}")
print(f"Remaining Conflicts: {final_conflicts}")
print(f"Iterations needed: {steps}\n")
print("Visual Representation:")
print_board(solution)
```

```
--- 8-Queens Simulated Annealing ---
Final State (Columns 0-7 Rows): [3, 7, 0, 4, 6, 1, 5, 2]
Remaining Conflicts: 0
Iterations needed: 1802

Visual Representation:
 .  .  Q  .  .  .  .  .
 .  .  .  .  .  Q  .  .
 .  .  .  .  .  .  .  Q
 Q  .  .  .  .  .  .  .
 .  .  .  Q  .  .  .  .
 .  .  .  .  .  .  Q  .
 .  .  .  .  Q  .  .  .
 .  Q  .  .  .  .  .  .
```